

Class- 11 Interrupt Task Program

```
/*
 * File:  main.c
 * Author:  sagar
 *
 * Created on 11 June, 2025, 2:27 PM
 */

// CONFIG
#pragma config FOSC = HS      // Oscillator Selection bits (HS oscillator)
#pragma config WDTE = OFF     // Watchdog Timer Enable bit (WDT disabled)
#pragma config PWRTE = OFF    // Power-up Timer Enable bit (PWRT disabled)
#pragma config BOREN = OFF    // Brown-out Reset Enable bit (BOR disabled)
#pragma config LVP = OFF      // Low-Voltage (Single-Supply) In-Circuit Serial Programming
                             // Enable bit (RB3 is digital I/O, HV on MCLR must be used for programming)
#pragma config CPD = OFF      // Data EEPROM Memory Code Protection bit (Data EEPROM
                             // code protection off)
#pragma config WRT = OFF      // Flash Program Memory Write Enable bits (Write protection off;
                             // all program memory may be written to by EECON control)
#pragma config CP = OFF       // Flash Program Memory Code Protection bit (Code protection off)

#include <xc.h>
#define _XTAL_FREQ 20000000

unsigned char value;

void main(void)
{
    // Configure RB0 (INT) as input
    TRISB |= (1 << 0); // RB0 input
    TRISC &= ~(1 << 0); // RC0 output
    TRISD &= ~(1 << 0); // RD0 output

    // Clear output ports
    PORTB = 0x00;
    PORTC = 0x00;
    PORTD = 0x00;

    // Setup interrupt
    OPTION_REG |= (1 << 6); // INTEDG: Interrupt on rising edge
    INTCON |= (1 << 4);     // INTE: Enable external interrupt on RB0/INT
    INTCON |= (1 << 6);     // PEIE: Peripheral interrupt enable
    INTCON |= (1 << 7);     // GIE: Global interrupt enable

    while(1)
    {
        // Main thread toggle RD0 every 1 second
        PORTD |= (1 << 0);
        __delay_ms(1000);
    }
}
```

```

    PORTD &= ~(1 << 0);
    __delay_ms(1000);
}
}

// Interrupt Service Routine
void __interrupt() ISR(void)
{
    if (INTCON & (1 << 1)) // Check INTF (bit 1) – External Interrupt Flag
    {
        value = PORTB;

        if (value & 0x01) // Check if RB0 is high
        {
            PORTC ^= (1 << 0); // Toggle RC0
        }

        INTCON &= ~(1 << 1); // Clear INTF flag
    }
}

```